



Chuyên đề: Thuật toán Tham lam (Greedy Algorithms)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

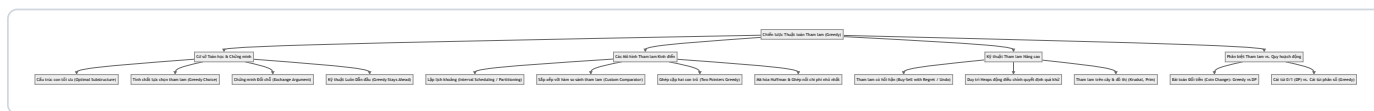
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Tổng quan & Bản đồ Tư duy (Mindmap)

Thuật toán Tham lam (Greedy Algorithm) là một trong những chiến lược thiết kế giải thuật nền tảng và phổ biến nhất trong Lập trình thi đấu (Competitive Programming). Nguyên lý cốt lõi của tham lam là: **"Tại mỗi bước ra quyết định, luôn chọn phương án tốt nhất ở thời điểm hiện tại (tối ưu cục bộ - Local Optimum) với kỳ vọng sẽ dẫn đến kết quả tốt nhất cho toàn bộ bài toán (tối ưu toàn cục - Global Optimum)."**

Tuy nhiên, tham lam là một "con dao hai lưỡi":

- Ưu điểm: Tốc độ thực thi cực nhanh (thường là $O(N)$ hoặc $O(N \log N)$ sau khi sắp xếp), cài đặt ngắn gọn, tiết kiệm bộ nhớ.
- Cạm bẫy: Rất dễ đưa ra chiến lược tham lam mang tính cảm tính nhưng sai về mặt toán học, dẫn đến lỗi Wrong Answer (WA) khó debug trong phòng thi.



2. Nền tảng Toán học & Kỹ thuật Chứng minh Tính Đúng đắn

Một bài toán muốn giải được bằng thuật toán tham lam bắt buộc phải thỏa mãn hai tính chất toán học:

- Tính chất cấu trúc con tối ưu (Optimal Substructure):** Nghiệm tối ưu của bài toán chứa nghiệm tối ưu của các bài toán con cấp thấp hơn.
- Tính chất lựa chọn tham lam (Greedy-choice Property):** Ta có thể đi tới nghiệm tối ưu toàn cục chỉ bằng cách đưa ra các lựa chọn tham lam cục bộ mà không cần phải quay lại xem xét các quyết định trong quá khứ hoặc dự đoán tương lai.

Để khẳng định một thuật toán tham lam là đúng đắn, trong lập trình thi đấu ta sử dụng hai kỹ thuật chứng minh kinh điển:

2.1. Kỹ thuật Đổi chỗ (Exchange Argument)

Đây là phương pháp chứng minh mạnh mẽ và thông dụng nhất cho các thuật toán tham lam dạng sắp xếp hoặc chọn tập hợp.

Khung chứng minh:

- Gọi $G = (g_1, g_2, \dots, g_k)$ là nghiệm do thuật toán tham lam tìm ra.
- Giả sử tồn tại một nghiệm tối ưu bất kỳ $OPT = (o_1, o_2, \dots, o_m)$ sao cho $OPT \neq G$.
- Xét vị trí đầu tiên i mà OPT khác với G ($o_i \neq g_i$).
- Ta thực hiện biến đổi $OPT \rightarrow OPT'$ bằng cách **tráo đổi (exchange)** hoặc thay thế phần tử o_i bằng phần tử tham lam g_i .
- Chứng minh rằng sau phép tráo đổi, giá trị hàm mục tiêu của OPT' :

$$\text{Cost}(OPT') \leq \text{Cost}(OPT) \quad (\text{với bài toán Min}) \quad \text{hoặc} \quad \text{Cost}(OPT') \geq \text{Cost}(OPT) \quad (\text{với bài toán Max})$$

Điều này chứng tỏ OPT' vẫn là một nghiệm tối ưu.

6. Lặp lại quá trình trao đổi hữu hạn lần, ta có thể biến đổi toàn bộ nghiệm OPT thành nghiệm G mà giá trị hàm mục tiêu không hề bị suy giảm.

7. **Kết luận:** Nghiệm tham lam G chính là một nghiệm tối ưu toàn cục.

2.2. Kỹ thuật Luôn Dẫn đầu (Greedy Stays Ahead)

Phương pháp này thường áp dụng cho các bài toán chọn số lượng phần tử nhiều nhất (ví dụ: Lập lịch khoảng không giao nhau).

Khung chứng minh:

- Giả sử thuật toán tham lam chọn được danh sách các phần tử $G = \{g_1, g_2, \dots, g_k\}$.
- Giả sử một nghiệm tối ưu bất kỳ chọn được danh sách $OPT = \{o_1, o_2, \dots, o_m\}$.
- Ta chọn một tiêu chí đo lường $f(x)$ (ví dụ: thời điểm kết thúc sớm nhất).
- Sử dụng quy nạp toán học để chứng minh rằng tại mọi bước $r \leq \min(k, m)$:

$$f(g_r) \text{ tốt hơn hoặc bằng } f(o_r)$$

- Từ đó suy ra thuật toán tham lam không bao giờ bị "chặn trước" nghiệm tối ưu, dẫn đến số phần tử chọn được $k \geq m$, do đó $k = m$ (tham lam đạt tối ưu).

3. Các Mô hình Tham lam Kinh điển trong Competitive Programming

3.1. Mô hình Lập lịch Khoảng (Interval Scheduling)

Bài toán 1: Chọn số lượng đoạn không giao nhau nhiều nhất

Cho N công việc, công việc i bắt đầu tại L_i và kết thúc tại R_i . Hai công việc không giao nhau nếu thời gian thực hiện của chúng không đè lên nhau. Hãy chọn số lượng công việc nhiều nhất có thể thực hiện.

- **Chiến lược tham lam đúng:** Luôn ưu tiên chọn công việc có **thời điểm kết thúc R_i nhỏ nhất**.
- **Giải thích trực giác:** Kết thúc càng sớm thì khoảng thời gian trống còn lại cho các công việc phía sau càng nhiều!
- **Thuật toán:**
 - Sắp xếp N đoạn theo R_i tăng dần.
 - Khởi tạo biến `last_end = -INF`, `count = 0`.
 - Duyệt qua từng đoạn: nếu $L_i \geq \text{last_end}$, chọn đoạn này và cập nhật `last_end = R_i`.
 - **Độ phức tạp:** $O(N \log N)$ (chỉ tốn thời gian sắp xếp).

```

struct Interval {
    long long l, r;
};

int maxNonOverlappingIntervals(vector<Interval>& intervals) {
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval& b) {
        return a.r < b.r; // Sắp xếp theo thời điểm kết thúc tăng dần
    });

    int count = 0;
    long long last_end = -2e18;
    for (const auto& iv : intervals) {
        if (iv.l >= last_end) {
            count++;
            last_end = iv.r;
        }
    }
    return count;
}

```

Bài toán 2: Phủ toàn bộ đoạn $[0, L]$ bằng số đoạn ít nhất

Cho N đoạn $[L_i, R_i]$ và một khoảng mục tiêu $[0, L]$. Tìm số lượng đoạn ít nhất để hợp của chúng phủ kín $[0, L]$.

- **Chiến lược tham lam đúng:**
- Sắp xếp các đoạn theo đầu mút trái L_i tăng dần.
- Duy trì vị trí hiện tại đã phủ được là `current_covered = 0`.
- Tại mỗi bước, xét tất cả các đoạn có $L_i \leq \text{current_covered}$, chọn đoạn có **đầu mút phải R_i xa nhất**.
- Cập nhật `current_covered = max_r`. Lặp lại cho đến khi `current_covered  $\geq L$` .
- **Độ phức tạp:** $O(N \log N)$.

3.2. Mô hình Sắp xếp với Hàm So sánh Tự định nghĩa (Custom Comparator via Exchange Argument)

Nhiều bài toán quy về việc sắp xếp một dãy phần tử theo một trật tự tối ưu sao cho hàm mục tiêu toàn cục đạt cực trị.

Bài toán Ghép số lớn nhất (Largest Number)

Cho danh sách các số nguyên không âm (dưới dạng xâu ký tự). Hãy sắp xếp và ghép các số đó lại để tạo thành một số nguyên có giá trị lớn nhất.

Ví dụ: `["3", "30", "34", "5", "9"]` → Ghép thành `"9534330"`.

- **Phân tích hàm so sánh:**
- Xét hai xâu A và B . Nếu ghép A trước B , ta thu được xâu $A + B$. Nếu ghép B trước A , ta thu được xâu $B + A$.
- Ta muốn A đứng trước B khi và chỉ khi $A + B > B + A$ (theo thứ tự từ điển).
- **Tính chất bắc cầu (Transitivity):**
Quan hệ $A + B > B + A$ thỏa mãn tính chất phản đối xứng và bắc cầu (Strict Weak Ordering), đảm bảo thuật toán `std::sort` chạy chính xác và ổn định.

```
string largestNumber(vector<string>& nums) {
    sort(nums.begin(), nums.end(), [](const string& a, const string& b) {
        return (a + b) > (b + a);
    });
    if (nums[0] == "0") return "0";
    string res = "";
    for (const string& s : nums) res += s;
    return res;
}
```

3.3. Mô hình Lập lịch Trừng phạt (Smith's Rule / Weighted Completion Time)

Cho N công việc, công việc i tốn thời gian thực hiện t_i và có trọng số trừng phạt w_i . Nếu công việc i hoàn thành tại thời điểm C_i , chi phí trừng phạt là $w_i \cdot C_i$. Hãy sắp xếp thứ tự thực hiện để tổng chi phí trừng phạt $\sum_{i=1}^N w_i C_i$ nhỏ nhất.

Chứng minh bằng Exchange Argument:

Xét hai công việc A và B đứng cạnh nhau trong lịch trình. Giả sử trước đó đã trôi qua thời gian T :

- Nếu xếp A trước B :

$$\text{Cost}_{AB} = w_A(T + t_A) + w_B(T + t_A + t_B)$$

- Nếu xếp B trước A :

$$\text{Cost}_{BA} = w_B(T + t_B) + w_A(T + t_B + t_A)$$

- Hiệu chi phí:

$$\text{Cost}_{AB} - \text{Cost}_{BA} = w_B t_A - w_A t_B$$

- Để A đứng trước B tốt hơn:

$$w_B t_A < w_A t_B \iff \frac{t_A}{w_A} < \frac{t_B}{w_B} \iff \frac{w_A}{t_A} > \frac{w_B}{t_B}$$

- **Quy tắc tham lam (Smith's Rule):** Luôn sắp xếp các công việc giảm dần theo tỷ số $\frac{w_i}{t_i}$.

4. Kỹ thuật Tham lam Nâng cao: Tham lam có Hối hận (Buy-Sell with Regret)

Trong các bài toán khó trên Codeforces (Div. 1/Div. 2 C-E), chiến lược tham lam ngây thơ (chọn ngay phương án tốt nhất mà không thể hoàn tác) thường thất bại vì lựa chọn ở hiện tại có thể làm mất đi cơ hội tốt hơn ở tương lai.

Kỹ thuật **Tham lam có hối hận (Regret Greedy / Undo Greedy)** giải quyết vấn đề này bằng cách:

1. **Cứ chọn tạm thời** phần tử ở hiện tại.
2. Lưu lựa chọn đó vào một cấu trúc dữ liệu (thường là **Min-Heap** / `std::priority_queue`).
3. Khi gặp một cơ hội tốt hơn trong tương lai mà nguồn lực bị giới hạn, ta **"hối hận"**: lấy phần tử tệ nhất đã từng chọn trong Min-Heap ra để hủy bỏ (hoàn tác) và thay thế bằng phần tử mới!

Dòng thời gian sự kiện:

- [Bước 1] --> Chọn A (Lưu vào Min-Heap)
- [Bước 2] --> Chọn B (Lưu vào Min-Heap)
- [Bước 3] --> Hết tài nguyên! Gặp C tốt hơn A rất nhiều!
==> Hối hận: Pop A ra khỏi Heap, nạp C vào!

Bài toán Điển hình: Potions (Codeforces 1526C)

Bạn đi qua n lọ thuốc. Lọ thứ i làm thay đổi lượng máu của bạn một lượng a_i (có thể âm hoặc dương). Bạn bắt đầu với 0 máu và máu không bao giờ được phép < 0 . Bạn muốn uống được nhiều lọ thuốc nhất có thể.

- **Thuật toán Tham lam có hối hận:**
- Duy trì `cur_health = 0` và một Min-Heap lưu các lọ thuốc âm đã uống.
- Khi gặp $a_i \geq 0$: Luôn uống ngay (`cur_health += a_i`).
- Khi gặp $a_i < 0$:
 - Cứ tạm thời uống: `cur_health += a_i`, đẩy a_i vào Min-Heap.
 - Nếu `cur_health < 0`: Ta buộc phải "hối hận" không uống lọ thuốc làm mất nhiều máu nhất từng uống $\implies$ Lấy phần tử nhỏ nhất từ Min-Heap ra (`cur_health -= heap.top()`, `heap.pop()`).
- **Độ phức tạp:** $O(N \log N)$ (thay vì $O(N^2)$ của DP thông thường).

5. Trace Thuật toán Từng bước (Step-by-Step Execution)

Xét bài toán Potions với dãy 6 lọ thuốc:

$$A = [4, -4, 1, -3, 1, -3]$$

Bước i	Lọ thuốc a_i	Hành động	Máu hiện tại (<code>cur_health</code>)	Min-Heap (các số âm đã uống)	Số lọ uống được
1	+4	Uống ngay	4	$\emptyset$	1
2	-4	Tạm uống	$4 + (-4) = 0 \geq 0$	$\{-4\}$	2
3	+1	Uống ngay	$0 + 1 = 1$	$\{-4\}$	3
4	-3	Tạm uống	$1 + (-3) = -2 < 0 \implies$ Hối hận! Bỏ -4, máu thành $-2 - (-4) = 2$	$\{-3\}$	3
5	+1	Uống ngay	$2 + 1 = 3$	$\{-3\}$	4
6	-3	Tạm uống	$3 + (-3) = 0 \geq 0$	$\{-3, -3\}$	5

Kết quả tối ưu: Uống được 5 lọ thuốc (chỉ bỏ qua lọ thuốc thứ hai mang giá trị -4).

⚠ 6. Cạm bẫy Phòng thi & Kỹ thuật Phân biệt Tham lam vs. DP (Traps & Pitfalls)

1. **Bẫy bài toán Đổi tiền (Coin Change Trap):**
2. Với hệ mệnh giá tiền xu chuẩn (như 1, 2, 5, 10, 20, 50, 100): Thuật toán tham lam (luôn chọn đồng xu lớn nhất $\leq$ số tiền còn lại) luôn cho số xu nhỏ nhất.

3. Với hệ mệnh giá bất kỳ (ví dụ $\{1, 3, 4\}$ và cần đổi số tiền 6):
 - Tham lam: Chọn 4 $\rightarrow$ còn 2 $\rightarrow$ chọn 1 $\rightarrow$ chọn 1 $\implies$ 3 đồng xu ($4 + 1 + 1$).
 - Nghiệm tối ưu (DP): $3 + 3 = 6 \implies$ chỉ cần **2 đồng xu!**
4. **Bài học:** Khi hệ thống không có cấu trúc Matroid hoặc không chứng minh được tính đối chố, bắt buộc phải dùng **Quy hoạch động (DP)**.
5. **Cạm bẫy toán tử so sánh (Strict Weak Ordering):**
6. Trong C++, khi viết hàm so sánh cho `std::sort`, hàm bắt buộc phải trả về `false` khi hai phần tử tương đương nhau (`a == b`).
7. Nếu viết `return a >= b;` thay vì `return a > b;`, chương trình có thể gặp lỗi **Segmentation Fault** hoặc **Undefined Behavior** trong quá trình sắp xếp.
8. **Tràn số khi nhân chéo trong hàm so sánh:**
9. Khi so sánh phân số $\frac{w_A}{t_A} > \frac{w_B}{t_B}$, ta biến đổi thành phép nhân chéo $w_A \cdot t_B > w_B \cdot t_A$.
10. Nếu $w, t \approx 10^9$, tích có thể lên tới 10^{18} gây tràn số `int` 32-bit. Luôn ép kiểu `1LL * w_A * t_B` hoặc dùng `long long`.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.